**Disciplina: Linguagem de Programação I**
**Semestre: 2026.2**
**Professor: Derzu Omaia**

# Projeto de Programação

**INSTRUÇÕES:**

1. **O projeto é individual, dupla ou trio.**
2. **A nota será composta por 70% do trabalho e 30% da apresentação.**
3. **Se usar IA, indique onde foi usada e use com moderação.**

## Temas:

1. Supermercado
2. Mercado
3. Banca Revista
4. Lanchonete
5. Doceria
6. Churrascaria
7. Churrasquinho
8. Sanduicheria
9. Restaurante
10. Bar
11. Quentinha/Marmita
12. Material de construção
13. Construtora
14. Consultório
15. Clínica
16. Livraria
17. Papelaria
18. Loja Roupas
19. Loja Genérica
20. Distribuidora
21. Artesanato
22. Artista Plástico
23. Controle estoque
24. Farmácia
25. Consertos Gerais
26. Marcenaria
27. Oficina
28. Equipadora de Carros
29. Costura
30. Entrega de água
31. Entrega de gás
32. Personal Trainer
33. Academia
34. Advocacia
35. Bijuteria
36. Cosméticos
37. Cartório
38. Imóveis/Imobiliária
39. Vendedor Autônomo
40. Assessoria Geral
41. Teatro
42. Agricultura.
43. Pecuária
44. Serigrafia
45. Floricultura
46. Táxi
47. Escola
48. Cabeleireiro
49. Manicure
50. Bodega
51. Revendedor Avon/Jequiti/Natura
52. Funerária
53. Outros

## Especificações:

1. Criar um sistema CRUD (Create, Read, Update, Delete) para cadastro de estoque/clientes/produtos ou realizar vendas relacionado a um dos temas sugeridos. O Menu deve possuir no mínimo essas opções:
   1. Inserir
   2. Listar todos
   3. Exibir um (pesquisar por nome)
   4. Alterar (pesquisar por nome)
   5. Remover (pesquisar por nome)
   6. Exibir Relatório
   7. Sair

   Obs: faça um método para pesquisar por nome (com espaço), e reuse ele nas opções, exibir um, alterar, remover. Este método deve retornar o índice do objeto, deve receber como parâmetro o string do nome a ser buscado. Sugestão de assinatura do método:

   ```
   int Pesquisar(std::string nome);
   ```

   A pesquisa deve aceitar o nome completo, ou apenas o início do nome.

   Obs2: Caso o seja encontrado mais de um objeto com o mesmo nome, deve-se exibir esses objetos e seus índices e selecioná-los através de seu índice.

2. Faça a modelagem das classes que serão utilizadas no sistema utilizando diagrama UML de classe. Cada classe deve ser colocada em um arquivo .h (ou hpp) separado, se quiser pode criar um .cpp para cada classe também.

3. O objeto principal deve ter pelo menos 4 atributos privados, entre eles um que seja do tipo de outra classe criada pela equipe (**composição**). Ex.: Endereço, Data, etc.

4. Utilize **herança** com pelo menos 1 superclasse e 2 subclasses. Os atributos da superclasse devem ser privados ou protected, e os atributos das subclasses devem ser privados.

5. Utilize uma lista/vetor com **polimorfismo**.

7. Implemente pelo menos um método que **sobrescreva algum método** da superclasse.

8. Utilize uma classe que vai gerenciar as operações CRUD. Não faça tudo na main();

9. Gerar relatório com um resumo das informações do sistema. Ex: Relatório de vendas, relatório de estoque, relatório de clientes. No relatório devem ser exibidas informações como, quantidade de elementos cadastrados, valor total, entre outros. Essa opção não é a mesma de “2. Listar todos”.

10. O programa deve rodar indefinidamente, até que o usuário escolha sair.

11. Todos os dados armazenados no programa devem ser salvos em arquivo, para que ao sair do programa os dados não se percam. E ao abrir o programa novamente os dados salvos devem ser carregados. **Utilize apenas um arquivo para salvar os dados polimórficos de mais de uma classe.**
    **Obs:** Carregar o arquivo automaticamente ao entrar na aplicação. Salvar em arquivo ao finalizar o programa. **Todas as operações de inserção, exibição, edição e remoção devem ser realizadas no vetor de objetos, e não diretamente no arquivo.**

**Entrega:**

- Apresentação rápida (até 10min) durante a aula;
- Entregar código fonte (arquivo zip ou link do repositório) e Diagrama de Classes;
- Data: 02/11/2026